Report: Assignment 2 - Factory Method & Abstract Factory

Notification System

by Abraimov Yeskendir, SE-2502

1. Introduction

For this assignment I built a small notification service: it sends messages over Email, SMS and Push, and styles them for a brand (Corporate or Marketing/Promo) before sending. The task splits into two parts, so I used two patterns.

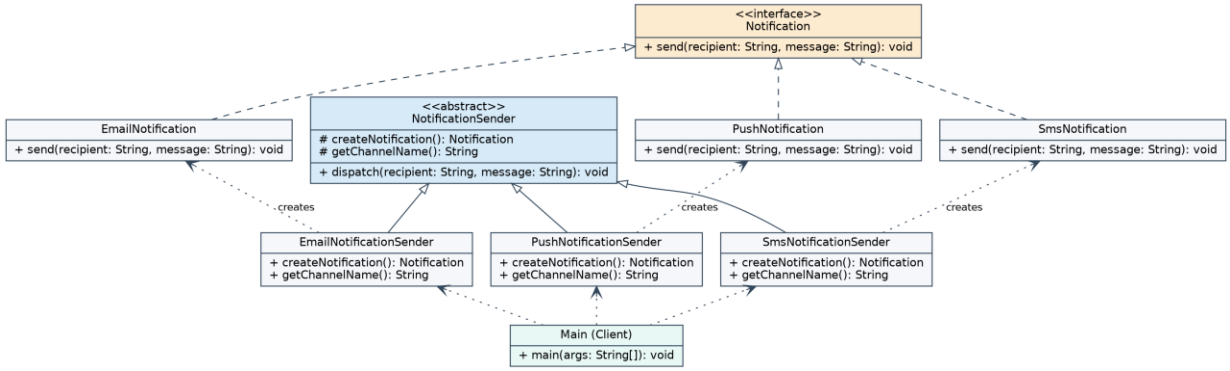
Picking a channel is Factory Method. The sending logic - log, create the transport, send it, log again - is the same for every channel, only the concrete Notification differs, so each Concrete Creator just decides which one to create.

Styling a message is Abstract Factory. A header, a body and a button all need to match the same brand, so one factory (CorporateBrandFactory or MarketingPromoFactory) produces all three together, instead of picking each part separately and risking a mismatch.

A Client class connects the two: it uses a BrandFactory to build the styled text, then passes the result to a NotificationSender, which delivers it through whichever channel was chosen.

2. UML Class Diagrams

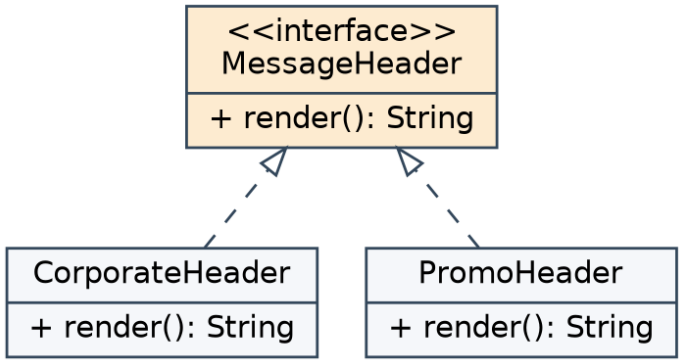
2.1 Part A - Factory Method

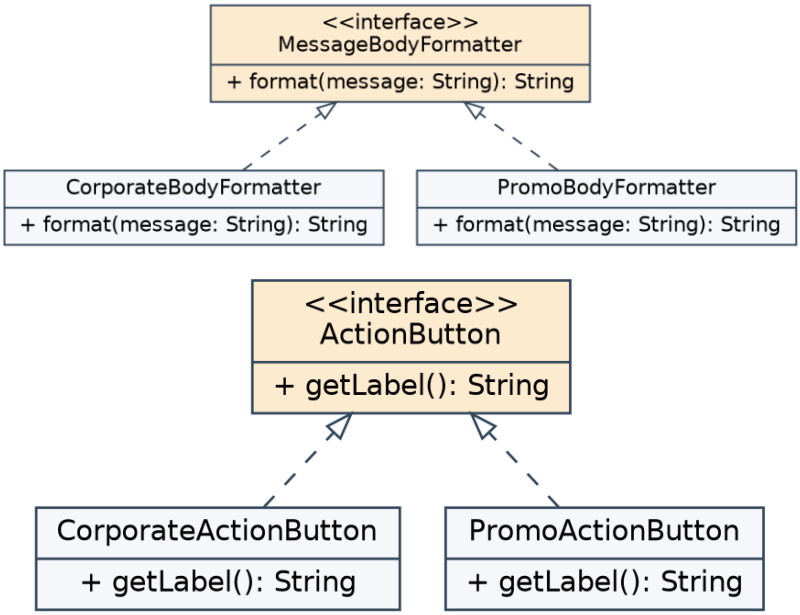


Notification - Product and NotificationSender - Creator

2.2 Part B - Abstract Factory

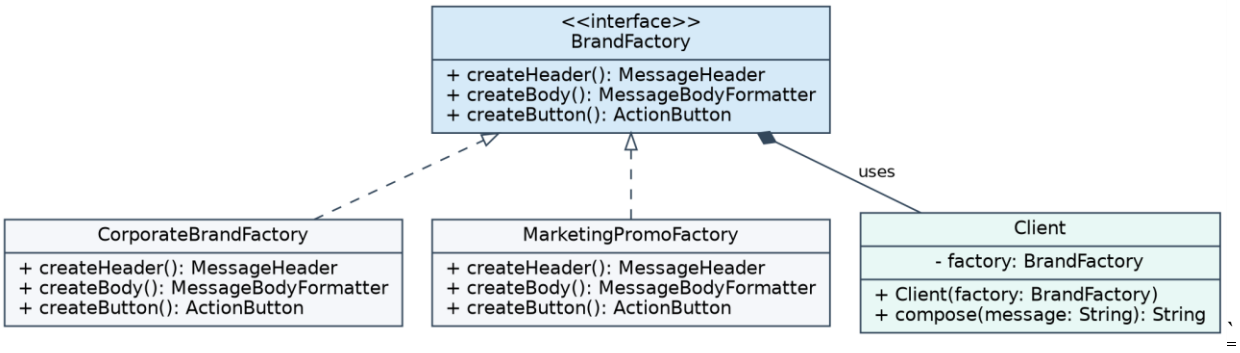
Abstract Products and their two concrete implementations (Corporate / Promo):





Product Family

Abstract Factory, its two concrete factories, and the Client that consumes them:



Abstract Factory with Concrete Factory

3. Clean Code Principles Applied

1. Command–Query Separation

Clean Code, Ch. 3 argues a function should either perform an action or answer a question, never both. Notification.send() is a command - it causes a side effect and returns void. MessageHeader.render(), MessageBodyFormatter.format() and ActionButton.getLabel() are queries - each hands back a String for the caller to use, with no printing inside them. Early on I wrote render() as void and printed from inside it, which broke this separation and also made the result impossible to reuse in Client.compose().

Before (query with a hidden side effect, doesn't compile against the String return type):

```
public String render(){
    System.out.println("Dear Customer,");
}
```

After - CorporateHeader.java (pure query, returns its result):

```
@Override
public String render() {
    return "Dear Customer, ";
```

```
}
```

2. Meaningful Names

Names should say what a method does and, where relevant, what it returns. ActionButton first had a method named link(String text) - a name that suggests a URL and a parameter that suggested it needed outside data, when a button label is really just a fixed piece of text. Renaming it to getLabel() (no parameter) made the interface self-explanatory.

Before - ActionButton.java:

```
public interface ActionButton {
    String link(String text);
}
```

After - ActionButton.java:

```
public interface ActionButton {
    String getLabel();
}
```

3. Small Functions, One Level of Abstraction (No Duplicated Logic)

NotificationSender.dispatch() holds every piece of behaviour that is common to all channels - the before/after logging and the call sequence - exactly once, instead of that logic being copied into EmailNotificationSender, SmsNotificationSender and PushNotificationSender separately. Each Concrete Creator only supplies the two things that actually differ: which Notification to build and what the channel is called.

Before (Creator hard-codes one concrete product, defeating the pattern):

```
public void dispatch(String recipient, String message){
    System.out.println("Preparing to send via " + getChannelName() + "...");

    Notification notification = new EmailNotification();
    notification.send(recipient, message);

    System.out.println("Notification sent via " + getChannelName() + ".");
}
```

After - NotificationSender.java (delegates to the factory method, reused by all three senders):

```
public abstract class NotificationSender {
    protected abstract Notification createNotification();
    protected abstract String getChannelName();

    public void dispatch(String recipient, String message) {
        System.out.println("Preparing to send via " + getChannelName() + "...");

        Notification notification = createNotification();
        notification.send(recipient, message);

        System.out.println("Notification sent via " + getChannelName() + ".");
    }
}
```

4. No Conditional / Type-Checking Logic in Client Code

The assignment specifically warns against a single switch/if-else chain picking the product type in client code. Neither Client.compose() nor the driver in Main uses any conditional to decide which concrete Header, Formatter, Button, or NotificationSender to use - the choice is made once, at object-construction time, by picking which factory or which sender subclass to instantiate. From that point on the code only calls interface methods (render(), format(), getLabel(), dispatch()) polymorphically.

Client.java - works only through BrandFactory and the abstract product interfaces:

```
public class Client {
    private final BrandFactory factory;

    public Client(BrandFactory factory) {
        this.factory = factory;
    }

    public String compose(String message) {
        MessageHeader header = factory.createHeader();
        MessageBodyFormatter formatter = factory.createBody();
        ActionButton button = factory.createButton();

        String headerText = header.render();
        String bodyText = formatter.format(message);
        String buttonText = button.getLabel();
        return headerText + "\n" + bodyText + "\n" + buttonText;
    }
}
```

5. Single Responsibility Principle

NotificationSender only knows how to log and deliver a message through one channel; it has no idea brands or templates exist. Client (together with BrandFactory) only knows how to assemble a brand-consistent message; it has no idea how that message eventually gets delivered. Keeping these two responsibilities in separate classes means a channel can be added or a brand style can be redesigned without touching the other side - each class has exactly one reason to change.

Main.java - the two responsibilities are composed, not merged, at the call site:

```
BrandFactory promoFactory = new MarketingPromoFactory();
Client promoClient = new Client(promoFactory);
String promoMessage = promoClient.compose("Hello");

NotificationSender testSms = new SmsNotificationSender();
testSms.dispatch("Yeska", promoMessage);

BrandFactory factory = new CorporateBrandFactory();
Client client = new Client(factory);
String finalMessage = client.compose("Whats up?");

NotificationSender testEmail = new EmailNotificationSender();
testEmail.dispatch("Amir", finalMessage);
```

4. Conclusion

Working through both patterns in the same project made the difference between them concrete rather than definitional. Factory Method earned its place wherever I only ever needed one object at a time and the only question was which concrete class - sending a notification never involves more than one Notification object, so a single overridden method was enough. Abstract Factory earned its place wherever several objects had to agree with each other - a header, a body and a button that must all match the same brand, which a Factory

Method alone cannot guarantee, since nothing stops a caller from mixing a Corporate header with a Promo button unless a single factory hands out the whole set together.

In general, I'd reach for Factory Method when a class hierarchy needs to create exactly one kind of related object and the concrete type is decided once, high up in the hierarchy. I'd reach for Abstract Factory as soon as ‘decided once’ needs to apply to more than one product at a time - whenever a wrong combination across products would actually be a bug, not just a style choice.

5. GitHub Repository

Repository: https://github.com/Naitskaut/SDP